Dataintegritet og automatisering

Indhold

Faglige mål 2

Målpind som dagens øvelse dækker 2

Baggrund 3

Øvelse 4

1. Vælg data strategi 4
2. Implementer seeding function i din app 4
3. Dataintegriteten 4
4. Constraints 4
5. Automatisering 5
6. Data historic/log 5

Hvordan du bliver bedømmet 5

Faglige mål

Målpind som dagens øvelse dækker

10. *Eleven kan anvende avancerede SQL-kommandoer som parametre, kontrolstrukturer, rettighedsstyring o.l.*
11. **Eleven kan redegøre for baggrunden for konsistenskrav, referenceintegritet, relationer og forskellige constraints.**
12. *Eleven kan oprette, redigere og slette Stored Procedures med et administrations- og databehandlingsformål.*
13. ***Eleven kan oprette og administrere Triggers med henblik på at holde databasen konsistent.**
14. *Eleven kan træffe korrekte forholdsregler til at imødegå SQL Injection.*
15. *Eleven kan oprette og administrere indexes så der opnås optimal performance.*
16. *Eleven kan udføre performance-måling på en stor database.*
17. **Eleven kan normalisere en database.**

Baggrund

En database skal ikke blot kunne **gemme data** og være normaliseret.

En database skal også kunne være med til at sikre, at data er **korrekte, konsistente** og **følger de regler, der gælder for systemet (**forretningsregler**)**. Man gøre dette med bl.a.:

- Dataintegriteten (forretningsregler)
- Constraints
- Identificér situationer, hvor databasen skal reagere automatisk bl.a. med triggers og stored procedures.
- Data historik eller log.

For dagens øvelse skal du tage udgangspunkt i den database, du udviklede under forrige øvelse.

Øvelse

Genbrug din .NET-applikation fra forrige øvelse.

1. Vælg data strategi

Du skal tag udgangspunkt i den database, du udviklede i øvelse 1. Du vælger selv hvilket af følgende database teknologi du vil anvend til kommunikation med din database i din .NET-applikation:

- ADO.net
- Dapper
- Entity Framework Core
- *Blanding af en eller flere teknologier nævn foroven*.

2. Implementer seeding function i din app

Opret en seeding function(er) i din app og insæt i din database nogle test data så du kan gennemføre kommende øvelser. (*Bruger indtast data kommer sener*)

3. Dataintegriteten

3.2. Undersøg hvad **forretningsregler** er for databaser. Der vil typisk altid være regler for **hvad der er tilladt i det konkrete system/domæne** som difineres som "forretningsregler". En Console App kan sagtens bygges med "layered architecture", hvor **forretningsregler ikke bør ligge direkte i databasekoden**.

- *En **simple** Layered architecture er tilstrækeligt (OOP inkapsulering) fordi dependency injection er ikke default implementeret i konsol applikationer. Elles vil dependency injection være primært bindeled i en mere advanceret Layered architecture.*

Difiner mindst 4 forretningsregler for din database, **en for hver alfabet i CRUD**.

3.3. Reflektér over din løsning.
Du vil blive spurgt ind til forretningsregler for din database og hvorfor og hvorledes dette implementeres. Formålet er at vurdere, hvor godt du forstår forretningsregler og dens relation til databaser. Vurdering anvendes som en del af det samlede vudering ved karaktergivning.

4. Constraints

4.1. Constraints er ligesom forretningsregler, men håndhæves af selve databasen med SQL:

- En **constraint** er en databasefunktion, der kan bruges til at **håndhæve regler.**
- En **forretningsregl** i applikationen kan også håndhæve forretningsreglen.

Undersøg deres forskellen og identificér mindst et område efter egen valg hvor constraints vil være nyttige og implementer dette.

4.2. Reflektér over din løsning.
Du vil blive spurgt ind til, hvilke regler du har identificeret for dine data, hvordan du har fået databasen til at håndhæve reglerne, og hvorfor du har valgt netop disse løsninger. Vurdering anvendes som en del af det samlede vudering ved karaktergivning.

5. Automatisering

5.1. Undersøg begrebet "trigger" for database og hvordan den relater til begrebet "automatisering" for database. Identificer herefter områder i din data, hvor automatisering med trigger vil være hjælpsomme og implementer dette.

Selv om trigger kan skrives direkt i databasen, skal du dog til denne øvelse opret din trigger med kode i din .NET app og med kode gem din trigger i din databasen.

5.2. Reflektér over din løsning.
Du vil blive spurgt ind til, hvad trigger er og hvordan den virker mod dine data, samt hvordan dette implementers i koden efter det stilled krav og hvorfor du har valgt netop disse løsninger. Vurdering anvendes som en del af det samlede vudering ved karaktergivning.

6. Data historic/log

6.1. Du skal implementere logning af operationer mod din database på en måde, der kan fungere som datahistorik. Kravet er, at du skal logge mod en fil:

```
LogID | Tidspunkt           | Bruger | Operation | Tabel  | RecordID | GamleData | NyeData
------+---------------------+--------+-----------+--------+----------+-----------+---------
  17  | 2026-09-21 20:14:32 | peter  | UPDATE    | Kunde  | 42       | ...       | ...
```

6.2. Reflektér over din løsning.
Du vil blive spurgt ind til, hvorfor historiske data og logning kan betragtes som en del af dataintegritet. Vurderingen anvendes som en del af den samlede vurdering ved karaktergivning
.

Hvordan du bliver bedømmet

- Du skal fremvis oprettelse af din database ud fra din .sql.
- Opgave indeholder en "refleksion" øvelse.
  - Du bliver bedømmet via spørgsmåler stillet med udgangspunkt i refleksionen af din øvelser.